Deploying Qwen3.8-27B in 12 GB of VRAM: Accuracy and Throughput Across Quantized Inference Stacks

Matthew Schwartz

Independent researcher ORCID 0009-0009-4171-7247 matthew.schwartz.research@gmail.com

August 29, 2026

Abstract

Quantized-inference studies usually report quality under a fixed engine or throughput under a fixed quantization, but a 12 GB GPU couples these choices: weights, cache, and engine buffers must fit together, and spilled weights are read from slower system RAM. We characterize this deployment problem for Qwen3.8-27B on an 80 W RTX 4080 Laptop GPU by measuring fit, throughput, and task performance for eight artifacts across GGUF/llama.cpp and EXL3/ExLlamaV3. Fully resident GGUFs decoded at 24.4–25.5 tokens/s, whereas partial offload reduced decode to 16.6–4.9 tokens/s. At approximately 9.5 GiB file footprint, EXL3/ExLlamaV3 remained fully resident while GGUF/llama.cpp offloaded six of 65 units; the hard-suite samples did not resolve an accuracy difference, but EXL3 generated 1.6× faster on MATH and achieved the highest observed correct responses per minute on MATH and HumanEval+. Across GGUF, the 2.16-effective-bpw artifact lost seven paired MATH items relative to the 2.49-effective-bpw artifact, while configurations from 2.49 to 5.22 effective bpw were not separated at these sample sizes; the original MATH parser also disagreed with the post-hoc normalizer unevenly across artifacts, so we report both scores separately. On this machine, joint artifact–engine selection kept a 27.4B model in 12 GB of VRAM and delivered correct responses faster than partial offload, with no hard-task difference resolved relative to the largest tested artifact; we did not test bf16 or Q8, or isolate engine from format.

1 Introduction

Quantized-inference work usually reports either quality under a fixed engine or throughput under a fixed quantization. A 12 GB discrete GPU couples the two: the file, cache, and engine buffers must fit together, and crossing that boundary moves weight reads from high-bandwidth VRAM to much slower system RAM. The practical question is therefore not simply "which quant has the best quality?" It is: *which artifact–engine pair returns correct answers fastest inside this fixed memory budget?*

We answer that question for Qwen3.8-27B, whose 27.36 billion non-MTP VLM parameters would require roughly 54.7 GB at bf16 before runtime overhead, on one 12 GB laptop GPU. In the measured run state, the right low-bit artifact and engine kept the model fully GPU-resident. Its scores on the small hard-task suites were not resolved from those of the largest tested artifact, and it returned more correct answers per minute than the larger deployments that placed weights in system RAM. This is the paper's central result: the machine did more useful work with a smaller, fully resident deployment than with the largest tested, partially offloaded artifact, which kept only 33 of 65 offload units on the GPU. More useful work here means more correctly scored responses per minute, not a resolved raw-accuracy advantage.

We first found the maximum GPU-resident layer count and measured prompt and decode throughput for a GGUF ladder. We then evaluated selected configurations on easy and harder task subsets, inspected MATH scoring failures item by item, added a GGUF control with nearly the same file footprint as EXL3, and compared the two recommended deployments on MMLU and long-context probes. We report nominal quantization, actual checkpoint size, GPU residency, task score, and wall-clock response rate separately.

Contribution and scope. The primary contribution is an end-to-end deployment map for a 27.4B model under a 12 GB laptop-GPU memory budget. It connects checkpoint footprint and engine-specific residency to controlled throughput, benchmark accuracy, and correct responses per minute. Fully resident GGUFs decoded at 24–25 tokens/s, whereas progressively offloaded GGUFs fell to 16.6–4.9 tokens/s. At matched file footprint, EXL3/ExLlamaV3 remained resident and generated faster than GGUF/llama.cpp without a resolved hard-suite score difference.

Two audits qualify that result. Nominal bit labels did not reliably predict whole-checkpoint size, and the post-hoc MATH normalizer accepted some surface forms that the original parser rejected. We therefore compare complete artifact–engine deployments rather than attributing the cross-stack result to trellis coding, and report the original-parser score beside a disclosed post-hoc-normalized score. The latter is a rescore of saved outputs, not an independent semantic ground truth.

The hard-suite subsets are small (n=20–25), every generation is a single sample at temperature 1.0, and engine and format cannot be separated in the cross-ecosystem comparison. Wilson 95% intervals accompany every accuracy. These data characterize the tested deployments; they do not establish model- or hardware-class generalities (§5).

2 Experimental Setup

2.1 Hardware

All measurements were taken on a single consumer laptop: NVIDIA GeForce RTX 4080 Laptop GPU (Ada Lovelace, SM 8.9), 12,282 MiB VRAM, capped at **80 W**; Intel Core i9-14900HX CPU; 62 GiB dual-channel DDR5; NVMe storage; Linux 7.0.0, driver 580.167.08. The desktop session held ∼900 MiB of VRAM, leaving a measured working envelope of roughly 10.5–11 GiB for weights, cache/state, and compute buffers. Power mode, memory clock, and sustained device clocks were not separately logged; the throughput results therefore describe this run state, not a normalized hardware limit.

Batch-one dense decode is usually memory-bandwidth-bound: each generated token's forward pass uses the active weight set, which is too large to remain in on-chip cache, so memory traffic scales approximately with active weight bytes and tok/s ≈ effective bandwidth/bytes per token. The laptop's GDDR6 is ∼432 GB/s on paper; our fitted effective GPU-resident weight-path bandwidth is 199 GB/s (§3.2). The fitted CPU-resident weight-path coefficient is 41 GB/s—roughly 5× lower. It absorbs memory-transfer and decode effects and is not a raw DDR5 specification. This two-bandwidth model is descriptive of the observed GGUF ladder; quant-family-specific CPU kernels account for its largest residuals.

2.2 Model

Qwen3.8-27B (released 2026-08-14) is an instruction-tuned vision–language model with 26.896B main-language parameters and 0.461B vision parameters (27.36B non-MTP parameters, rounded to 27.4B here). Its separate MTP head has 0.425B parameters. The model card reports a *hybrid* sequence mixer: only 16 of 64 layers are classical attention with a growing KV cache (4 KV heads × 256 head dim), while the other 48 are DeltaNet-style linear-attention layers [1]. Gated Delta Networks and Mamba provide the architectural background [2, 3]; the recurrent layers hold a

Table 1: Eight evaluated weight artifacts. Effective bpw = $8B/26{,}895{,}998{,}464$ for checkpoint weight-file total B, normalized by the exact main-language parameter count; it can exceed the nominal label because embeddings (248K vocabulary), norms, and (for EXL3) the 3-bit head quantize wider than the label. Downloaded 2026-08-25 from `unsloth/Qwen3.8-27B-GGUF`, `bartowski/Qwen3.8-27B-GGUF`, and turboderp's EXL3 conversions (`SC_2.00bpw_H3`). The optional unsloth MTP draft is listed separately and is not included in its target file size.

Artifact	Format	File	Eff. bpw	Note
`UD-IQ2_XXS`	GGUF i-quant, unsloth UD	6.77 GiB	2.16	smallest under test
`UD-IQ2_S`	GGUF i-quant, unsloth UD	7.80 GiB	2.49	largest full-GPU fit
`UD-Q2_K_XL`	GGUF k-quant, unsloth UD	9.15 GiB	2.92	
`IQ2_S` (bartowski)	GGUF i-quant	9.59 GiB	3.06[a]	MTP head baked in
`UD-IQ3_XXS`	GGUF i-quant, unsloth UD	10.18 GiB	3.25	
`UD-IQ3_S`	GGUF i-quant, unsloth UD	11.21 GiB	3.58	
`UD-Q4_K_XL`	GGUF k-quant, unsloth UD	16.35 GiB	5.22	largest quality anchor
EXL3 2.0 bpw	EXL3 trellis (QTIP-style)	9.50 GiB	3.03	nominal 2.0; head at 3 bpw
Optional MTP draft	GGUF Q4_0	1.28 GiB	—	separate for unsloth targets

[a] Includes the baked-in MTP head, so main-model bpw is lower.

constant-size state (∼150 MB total here). Verified KV cost is $2 \times 4 \times 256 \times 2\,\text{B} \times 16 = \mathbf{64\,KiB/token}$ at fp16 (∼38 KiB at q8_0) — 4× lower than a counterfactual 64-layer model using the same four-KV-head attention block in every layer. At the tested context lengths, KV-memory growth was therefore relatively cheap and *fitting was primarily a weights problem.* The release ecosystem also provides a 1.28 GiB multi-token-prediction (MTP) head; the tested llama.cpp build used it as an in-model draft [1, 4], an instance of speculative decoding [5, 6]. It also provides a 0.86 GiB vision projector (vision was not evaluated). These components are packaged differently: the unsloth GGUF release stores both separately, the bartowski GGUF embeds the MTP tensors, and the tested EXL3 checkpoint has no configured draft or vision path. The 248K-token vocabulary makes the text embeddings a large, hard-to-quantize fraction of every weight artifact — one reason nominal and effective bits-per-weight diverge (Table 1).

2.3 Engines

Two inference stacks: **llama.cpp** [4] built from master at `b1-1729ed5` with a user-local CUDA 12.6.3 toolchain (SM 8.9), serving GGUF artifacts with layer-wise CPU offload (`-ngl`); and **ExLlamaV3 1.4.3** [7] via TabbyAPI, serving EXL3 trellis-quantized safetensors. For this tested dense checkpoint, the stack used no supported dense-layer CPU weight-offload path, so the complete model had to fit in VRAM. Unless noted: context 8192, KV cache q8_0, flash attention on, the model card's recommended thinking-mode sampling (temperature 1.0, top-p 0.95, top-k 20) [1], and the chat template's `reasoning_effort` set to *medium*.

2.4 Artifacts

Table 1 lists the eight evaluated weight artifacts with their measured checkpoint weight-file total and *effective* bits per weight, computed as $8B/26{,}895{,}998{,}464$ for total weight-file bytes B and the exact main-language parameter count. We report effective bpw beside every nominal label throughout the paper; the two diverge by up to 52% and the difference determines which artifacts are valid footprint controls in §3.7.

Table 2: Controlled GGUF speed matrix (llama.cpp `b1-1729ed5`; ctx 8192, KV q8_0, flash-attn on; desktop session running). tg = `llama-bench` tg128; server-measured decode agreed within ±0.4 tok/s. EXL3 is excluded because no matching `tg128` measurement was collected; its workload measurements are reported separately in §3.7.

Config	File	GPU layers	tg tok/s	pp512	VRAM peak	MTP effect on tg
`UD-IQ2_XXS`	6.8 GiB	**65/65**	**25.5**	733	8.6–8.9 GiB	+21% → 30.9 (GPU draft); 48–50 math; −16% CPU draft
`UD-IQ2_S`	7.8 GiB	**65/65**	**24.4**	708	9.7–10.0 GiB	draft misses fit by ∼100 MiB
`UD-Q2_K_XL`	9.2 GiB	61/65	16.6	652	10.0–10.4 GiB	crashed (draft buffers, see text)
`IQ2_S` (bartowski)	9.6 GiB	59/65	13.8	580	9.5–10.2 GiB	not tested
`UD-IQ3_XXS`	10.2 GiB	54/65	11.3	608	10.4–10.8 GiB	crashed (draft buffers)
`UD-IQ3_S`	11.2 GiB	49/65	8.2	559	10.5–10.7 GiB	crashed (draft buffers)
`UD-Q4_K_XL`	16.4 GiB	33/65	4.9	361	10.5 GiB	wash (+8% prose, −8% code)
CPU floor (`UD-IQ2_S`)	7.8 GiB	0/65	3.3	4.1	—	—

2.5 Evaluation Protocol and Metrics

The fit-and-throughput campaign used a guarded harness (never launch unless estimated VRAM ≤ free−500 MiB and available RAM covers CPU-resident bytes +8 GiB; weights always mmap-backed) that runs, per artifact: a fit search for the maximum GPU-resident layer count at ctx 8192, `llama-bench` pp512/tg128 at that fit, and timed `llama-server` generations (prose and code prompts, MTP off/on). llama.cpp reports 65 offload units for this 64-block model because its count includes the output layer. Each result is written and flushed to JSONL immediately.

The accuracy campaign evaluated served models through the OpenAI-compatible endpoint on four suites: **GSM8K** [8] ×50, strict answer extraction; **HumanEval** [9] ×20, generated code executed against the official tests; **MATH-25**: 25 competition problems at MATH levels 4–5 [10], 8K-token generation budget, truncations scored wrong; and **HumanEval+** ×20 [11], execution against EvalPlus's ∼80× augmented test sets. Every accuracy in this paper carries a Wilson 95% interval in brackets. The subsets were sampled deterministically with seed 42; MATH-25 was stratified across all seven subjects, and HumanEval+ used the same 20 problems as HumanEval. A final validation compared the two selected deployments on a 200-item, 57-subject-stratified MMLU subset [12] and six synthetic needle contexts (8K/16K/32K, two insertion depths). For MATH, the *original-parser score* applies the answer extractor and acceptance rules used during the run. The *post-hoc-normalized score* applies a disclosed, deterministic normalizer to the same saved responses; it does not regenerate answers and is not an independent judgment of semantic correctness. At n=20–25 the intervals are wide. They reflect item sampling, not the additional variation from single stochastic generations.

3 Results

3.1 GPU Residency and Throughput

Table 2 and Figure 1 give the matrix. The full-residency boundary is the dominant feature; nominal labels and prompt processing add two important qualifications.

For llama.cpp/GGUF, the full-GPU boundary lies between 7.8 and 9.2 GiB in this run state. With the desktop session holding ∼900 MiB, only the 6.8 and 7.8 GiB artifacts fit all 65 layers. Everything larger sheds layers to system RAM and slows immediately. Within the GGUF ladder, footprint and consequent residency are the first-order predictors of speed. This boundary is engine-specific: ExLlamaV3 later fits a 9.50 GiB EXL3 checkpoint (§3.7).

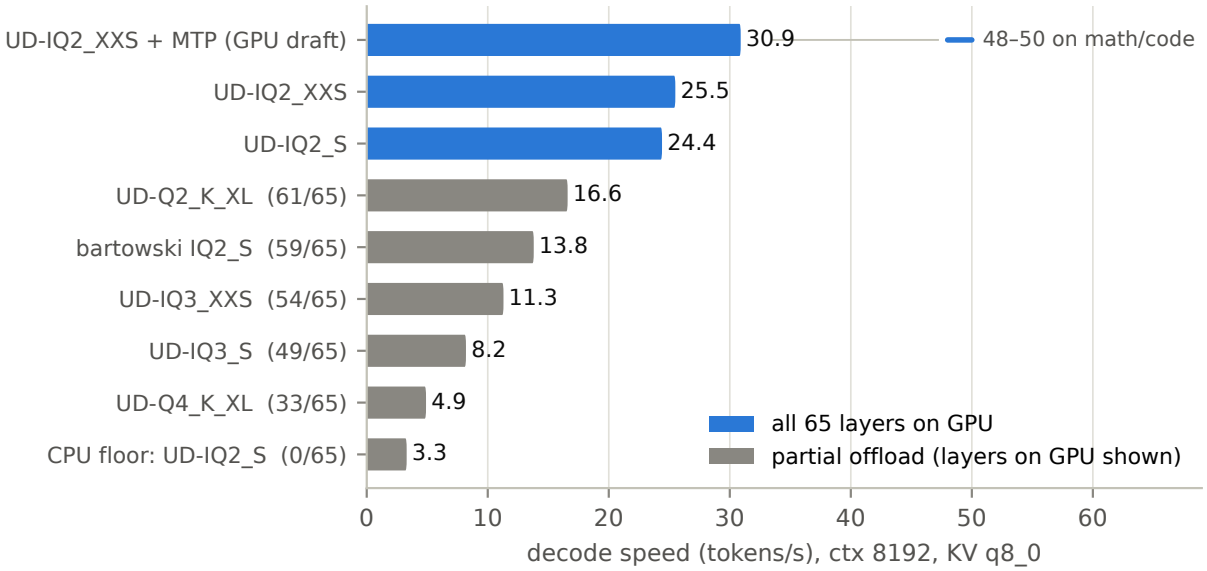

Figure 1: The measured speed ladder. Blue: all 65 layers GPU-resident; gray: partial offload (GPU layer count in the label). The `UD-IQ2_XXS`+MTP bar shows prose decode; the whisker marks its measured 48–50 tok/s on math/code content (§3.3). All base bars use the same `llama-bench` tg128 protocol; the MTP annotations are separate server measurements.

The nominal quant label does not determine footprint or speed. Unsloth's `UD-IQ2_S` (7.8 GiB, 2.49 effective bpw) runs 24.4 tok/s; bartowski's `IQ2_S` (9.6 GiB, 3.06 effective bpw) runs 13.8 tok/s. The files share a nominal label but not a size, tensor allocation, or packaging: the bartowski file also embeds MTP tensors. The observed speed difference is consistent with full versus partial residency; it does not isolate a quant-maker or allocation effect.

At 512 prompt tokens, GPU-heavy configurations ingest much faster than they decode. The controlled pp512 rates were 360–733 tok/s, while the CPU-only i-quant floor fell to 4.1 tok/s. These short-context microbenchmarks must not be extrapolated to 32K prompts; the deadline behavior in §3.7 shows why.

3.2 Offload Decay Follows a Two-Bandwidth Model

Figure 2 tests the bandwidth model of §2.1 against the matrix. Per-token time is linear in the two resident byte counts, with fitted effective path bandwidths of **199 GB/s** (GPU-resident bytes) and **41 GB/s** (CPU-resident bytes) — a 4.9× ratio, and the reason each shed layer costs so much. The fit's residual structure is itself informative: deviations concentrate where CPU dequantization cost differs by quant family, i.e., the "linear in bytes" rule needs a per-family CPU coefficient before it can be used predictively across formats. For ranking configs on one family it is already accurate to a few percent.

3.3 MTP Benefit Depends on Workload and Placement

On the controlled prose prompt, the MTP draft head increased decode throughput by 21% (25.5 → 30.9 tok/s) at temperature 1.0 with the draft fully on GPU, at a 41% acceptance rate. During the MATH run, the same deployed configuration produced 45–50 output tokens/s. That higher workload rate is consistent with greater acceptance on structured text, but no draft-off run was collected on the identical MATH prompts, so it is not a controlled 90% MTP effect. Likewise, the

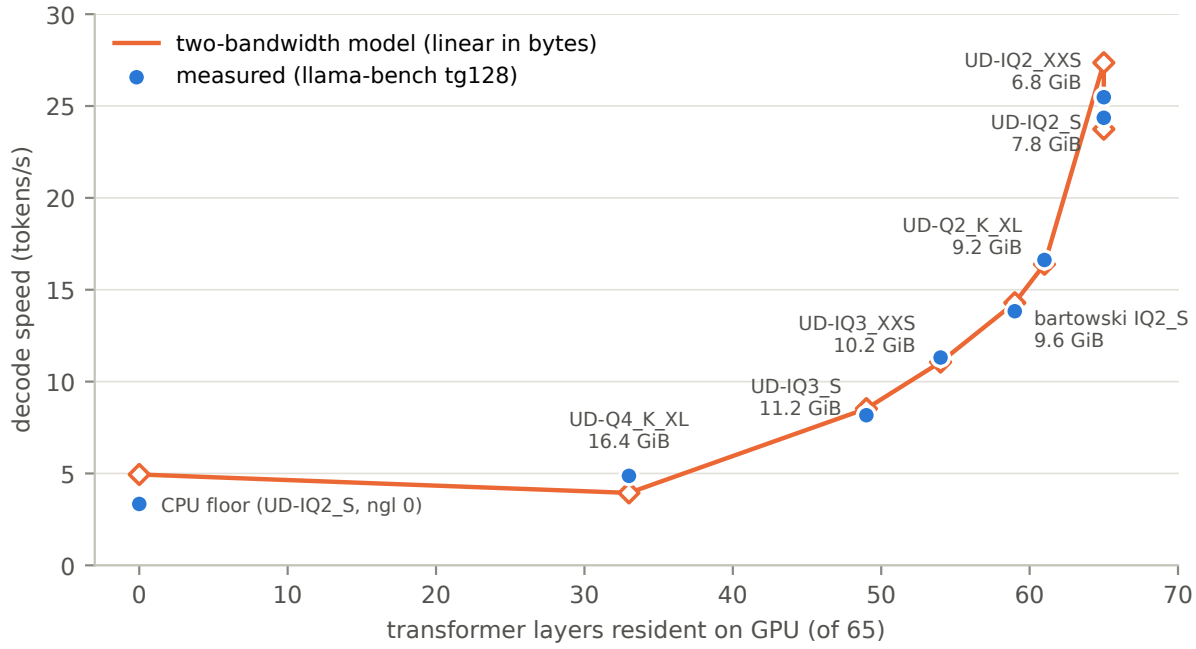

Figure 2: Decode speed vs. GPU-resident layer count (blue, measured), with the two-bandwidth model's per-config predictions (orange). The model, fitted for relative error over all eight points, is $1/\text{tok/s} = b_{\text{GPU}}/199\,\text{GB/s} + b_{\text{CPU}}/41\,\text{GB/s}$ where b are resident byte counts (file bytes split proportionally to layers). The six non-extreme measured states land within ±8%; the two extremes' deviations are consistent with a quant-family-dependent CPU term (i-quants dequantize expensively on CPU, so the ngl-0 floor runs 32% below the pure-bandwidth prediction, while k-quant `UD-Q4_K_XL` runs 24% above it).

9.6 s mean GSM8K item time for `UD-IQ2_XXS`+MTP versus 17.7 s for `UD-IQ2_S` is a deployment comparison, not an isolated MTP comparison: both the artifact and draft setting change.

Two operational caveats. First, with the draft on CPU, MTP was a *loss* (−16%) or a wash in the controlled prose runs; these results should not be carried over to other sampling regimes. Second, the draft's VRAM cost is treacherous: ∼2.8 GiB GPU-side when GPU-hosted, and still ∼**1.2 GiB** when nominally CPU-hosted (`-ngld 0`), because its context and compute buffers allocate on the GPU regardless. This silently crashed every tightly fitted mid-size config in our first sweep (the "crashed" rows of Table 2) until the harness guard learned to account for it.

The result is therefore configuration-specific: MTP can help substantially when the draft fits on GPU, but it is neither free in VRAM nor described by one workload-independent speedup.

3.4 Easy-Suite Accuracy Screen

Table 3 has little resolution among the larger artifacts: GSM8K estimates span 94–98%, and HumanEval estimates span 95–100% above 2.16 effective bpw. `UD-IQ2_XXS` has the lowest HumanEval estimate (80%), but all intervals are wide. The practical difference is time. Across these two suites, `UD-Q4_K_XL` used 4.6× the pooled wall-clock of `UD-IQ2_S` (7318 s/1606 s) and recorded two additional successes out of 70; no accuracy difference was resolved. Because these older suites cluster near the top of their scales, the next experiments use the fixed MATH-25 subset and HumanEval+ to seek greater separation.

Table 3: GSM8K (n=50) and HumanEval (executed, n=20) at reasoning-effort medium, with Wilson 95% CIs. Correct/min = number of successful items divided by summed logged HTTP-response wall time (local post-response scoring excluded); compare only within a suite.

Config	tok/s	GSM8K	HumanEval	correct/min	
				GSM8K	HE
`UD-IQ2_XXS`+MTP	30.9 (≈49 math)	96% [86.5, 98.9]	80% [58.4, 91.9]	**6.01**	1.66
`UD-IQ2_S`	24.4	94% [83.8, 97.9]	**100%** [83.9, 100]	3.18	**1.67**
`UD-Q2_K_XL`	16.6	94% [83.8, 97.9]	100% [83.9, 100]	2.26	1.53
`IQ2_S` (bartowski)	13.8	98% [89.5, 99.6]	95% [76.4, 99.1]	2.21	1.05
`UD-IQ3_XXS`	11.3	96% [86.5, 98.9]	100% [83.9, 100]	1.65	0.92
`UD-IQ3_S`	8.2	98% [89.5, 99.6]	100% [83.9, 100]	1.23	0.70
`UD-Q4_K_XL`	4.9	98% [89.5, 99.6]	100% [83.9, 100]	0.66	0.42

Table 4: MATH-25 (n=25, competition level 4–5): original parser score vs. alternate-normalizer score, with the number of upgraded items (value-equivalent under the alternate rules) and truncations (8K generation budget exhausted; wrong under both scorers). Wilson 95% CIs use the alternate score. Reasoning-effort medium except where noted. Two `UD-Q4_K_XL` rows lost their ground-truth field to a harness bug; the rescorer recovers them from the dataset.

Config (eff. bpw)	Original	Alternate	CI (alternate)	Upgraded	Trunc.
`UD-IQ2_XXS`+MTP (2.16)	12/25	16/25 (64%)	[44.5, 79.8]	4	3
`UD-IQ2_S` (2.49)	14/25	**23/25 (92%)**	[75.0, 97.8]	**9**	1
`UD-IQ2_S`, xhigh (2.49)	17/25	21/25 (84%)	[65.3, 93.6]	4	4
EXL3 2.0 bpw (3.03)	20/25	**24/25 (96%)**	[80.5, 99.3]	4	1
`IQ2_S` bartowski (3.06)	20/25	23/25 (92%)	[75.0, 97.8]	3	1
`UD-IQ3_XXS` (3.25)	22/25	**24/25 (96%)**	[80.5, 99.3]	2	0
`UD-Q4_K_XL` (5.22)	18/25	23/25 (92%)	[75.0, 97.8]	5	0

3.5 MATH Answer-Parser Sensitivity

The original MATH parser scored `UD-IQ2_S` at 14/25 and `UD-IQ3_XXS` at 22/25. Inspection of the saved responses found rejected strings that differed from the reference in answer tags, math delimiters, Unicode symbols, degree notation, brace shorthand, or an equation such as `x=5` where the key contained `5`.

After inspecting those failures, we wrote a deterministic alternate normalizer and applied it to every saved prediction. It accepts the declared equivalences, leaves truncations wrong, and leaves mechanically unresolved cases wrong. Because the rules were chosen post hoc, this score is not an independent measure of mathematical correctness and does not replace the original deployment parser. Table 4 and Figure 3 report both scores, with the rules and retained per-item scoring records available for audit. No inference was rerun for this analysis.

Accuracy across artifacts. Under the post-hoc normalizer, the estimates from 2.49 to 5.22 effective bpw are 92–96% and are not resolved from one another at this sample size. The 2.16-effective-bpw artifact scored 16/25, compared with 23/25 at 2.49 bpw on the same items: seven successes favored the 2.49-bpw artifact and none favored 2.16 bpw (paired exact p=0.016, exploratory and uncorrected). No GGUF was tested between 2.16 and 2.49 effective bpw, so the experiment identifies a difference between these artifacts, not a universal threshold.

Parser sensitivity. For `UD-IQ2_S`, the post-hoc normalizer accepted 9/25 responses rejected by the original parser; for `UD-IQ3_XXS`, it accepted 2/25. On the paired items, seven changed only for

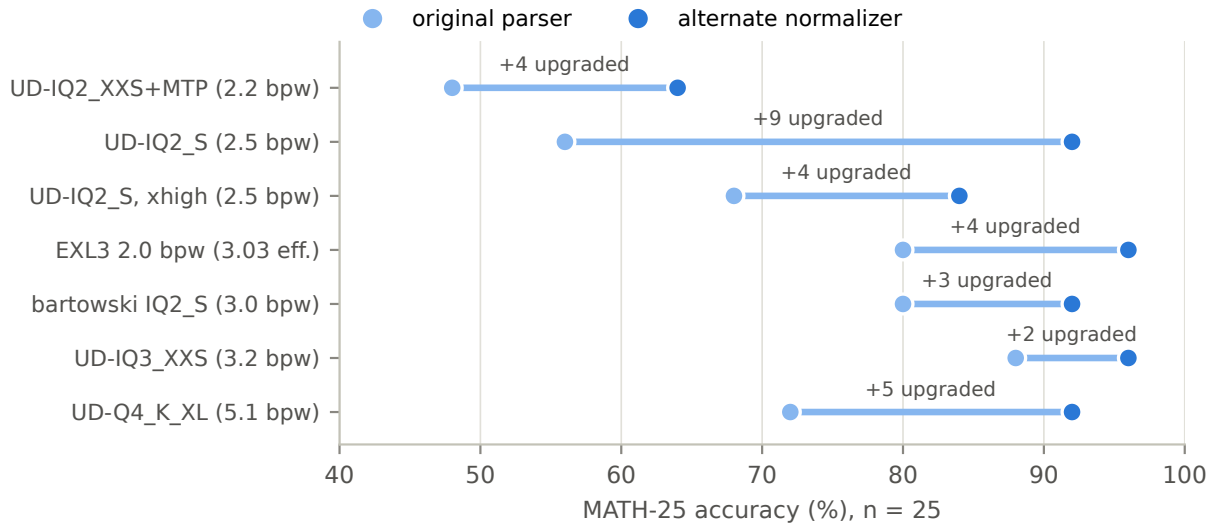

Figure 3: Original→alternate score changes on MATH-25 (n=25). The connector shows how many responses the post-hoc normalizer accepts that the original parser rejected; both endpoints remain visible because the rescore is not independent ground truth. The largest change is 9/25 at 2.49 effective bpw, versus 2/25 at 3.25 effective bpw.

IQ2_S and none only for IQ3_XXS (exact p=0.016, exploratory and uncorrected). This establishes that parser acceptance changed the apparent gap unevenly in these saved outputs. It does not establish that every newly accepted response is semantically correct, or that quantization generally harms answer formatting before reasoning. Parser-compatible output remains a real deployment capability, so both scores are retained.

Execution-scored code. Generated code is extracted from a fenced block and then executed, so it avoids symbolic-answer normalization but still treats a missing/malformed code block as failure. We did not post-hoc rescore those format failures; Table 5 reports the end-to-end parser and test outcome as deployed.

3.6 Hard-Suite Accuracy

Table 5 collects the hard suites. The 2.16-effective-bpw artifact has the lowest point estimate on both: 16/25 alternate MATH and 14/20 HumanEval+, compared with 23/25 and 18/20 for the 2.49-bpw artifact. Because every configuration saw the same items, paired tests are appropriate. The MATH comparison has seven IQ2_S-only successes and no IQ2_XXS-only successes (exact two-sided p=0.016); HumanEval+ has four and zero, respectively (p=0.125). Thus these data resolve the MATH loss but not the code-suite difference. We do not pool the two suites into a single significance test.

Above 2.16 effective bpw, alternate MATH estimates are 92–96% and HumanEval+ estimates are 90–95%; no difference is resolved at these sample sizes. That is evidence for a plateau within this experiment, not proof that the artifacts or formats are equivalent. It also reinforces why nominal "2-bit" is too coarse a label: the evaluated files bearing it span 2.16–3.06 effective bpw and produced materially different hard-suite point estimates.

3.7 Matched-Footprint Deployment Comparison

The EXL3 checkpoint is labeled 2.0 bpw, but its 9.50 GiB weight-file total is **3.03 effective bpw**: the transformer weights target roughly 2 bpw, while the output head is 3-bit and other tensors are

Table 5: Hard-suite summary (reasoning-effort medium). HumanEval+ is execution-scored (n=20), Wilson 95% CIs. Correct/min uses the alternate MATH-25 score and each run's summed logged HTTP-response wall time; post-response parsing and code execution are excluded. The xhigh HumanEval+ entry is the truncation-free re-run (§3.8).

Config (eff. bpw)	MATH-25 (alternate)	HumanEval+	correct/min	
			math	HE+
`UD-IQ2_XXS`+MTP (2.16)	64% [44.5, 79.8]	70% [48.1, 85.5]	0.69	1.99
`UD-IQ2_S` (2.49)	92% [75.0, 97.8]	90% [69.9, 97.2]	0.73	1.40
`UD-IQ2_S`, xhigh (2.49)	84% [65.3, 93.6]	80% [58.4, 91.9]	0.37	0.73
EXL3 2.0 bpw (3.03)	**96%** [80.5, 99.3]	90% [69.9, 97.2]	**0.83**	**2.27**
`IQ2_S` bartowski (3.06)	92% [75.0, 97.8]	90% [69.9, 97.2]	0.38	0.86
`UD-IQ3_XXS` (3.25)	**96%** [80.5, 99.3]	90% [69.9, 97.2]	0.33	0.73
`UD-Q4_K_XL` (5.22)	92% [75.0, 97.8]	**95%** [76.4, 99.1]	0.19	0.43

Table 6: Selected-deployment validation. MMLU used the same fixed 200-item, 57-subject-stratified subset; CIs are Wilson 95%. Correct/min uses logged HTTP-response wall time (post-response scoring excluded). Needle reports probes completed correctly before the client exhausted two 600-s HTTP attempts (one automatic retry), not pure retrieval accuracy.

Config	MMLU	Mean wall/item	Correct/min	Needle deadline success
`UD-IQ2_S`	175/200, 87.5% [82.2, 91.4]	36.41 s	1.44	4/6 [30.0, 90.3]
EXL3 2.0 bpw	182/200, 91.0% [86.2, 94.2]	25.42 s	2.15	6/6 [61.0, 100]

wider. It therefore is not a footprint-matched comparison with the 6.77 GiB, 2.16-effective-bpw GGUF. Crediting their quality difference to trellis coding would confound method with 40% more effective bits.

The direct footprint control is bartowski `IQ2_S`: 9.59 GiB and 3.06 effective bpw. On identical hard-suite subsets, EXL3 scored 24/25 versus 23/25 on alternate MATH and both scored 18/20 on HumanEval+. These small samples do not resolve a quality difference. They do expose a deployment difference: the EXL3–ExLlamaV3 stack was fully GPU-resident, whereas the bartowski-GGUF–llama.cpp stack offloaded six of 65 units. From the server logs on these same workloads, token-weighted generation throughput was 22.4 versus 13.7 tok/s on MATH (1.64×) and 30.7 versus 13.8 tok/s on HumanEval+ (2.22×). The rate varies with output length and workload, so we use the MATH result as the headline comparison: **at matched footprint, the fully resident EXL3 deployment generated** 1.6× **faster, and the available hard-suite samples did not resolve a score difference.**

The other measured comparisons clarify what this result does and does not identify:

- At similar effective bpw, EXL3 (3.03) and `UD-IQ3_XXS` (3.25) both scored 24/25 on alternate MATH and 18/20 on HumanEval+. We measured no quality-per-effective-bit advantage for either artifact at this point.
- Against `UD-IQ2_XXS` (2.16), EXL3 has higher point scores but also a 2.73 GiB larger file. This experiment cannot locate a trellis-specific quality boundary near 2.2 effective bpw.

This is the concrete sense in which the 12 GB system exceeded a naive capacity-based expectation: a 9.50 GiB checkpoint remained resident and ran at useful speed, while a nearly identical-size GGUF offloaded weights to RAM. Because format and engine changed together, the experiment compares complete deployments and cannot attribute the difference to trellis coding alone.

On paired MMLU items, both were correct on 171, only `UD-IQ2_S` on four, only EXL3 on 11,

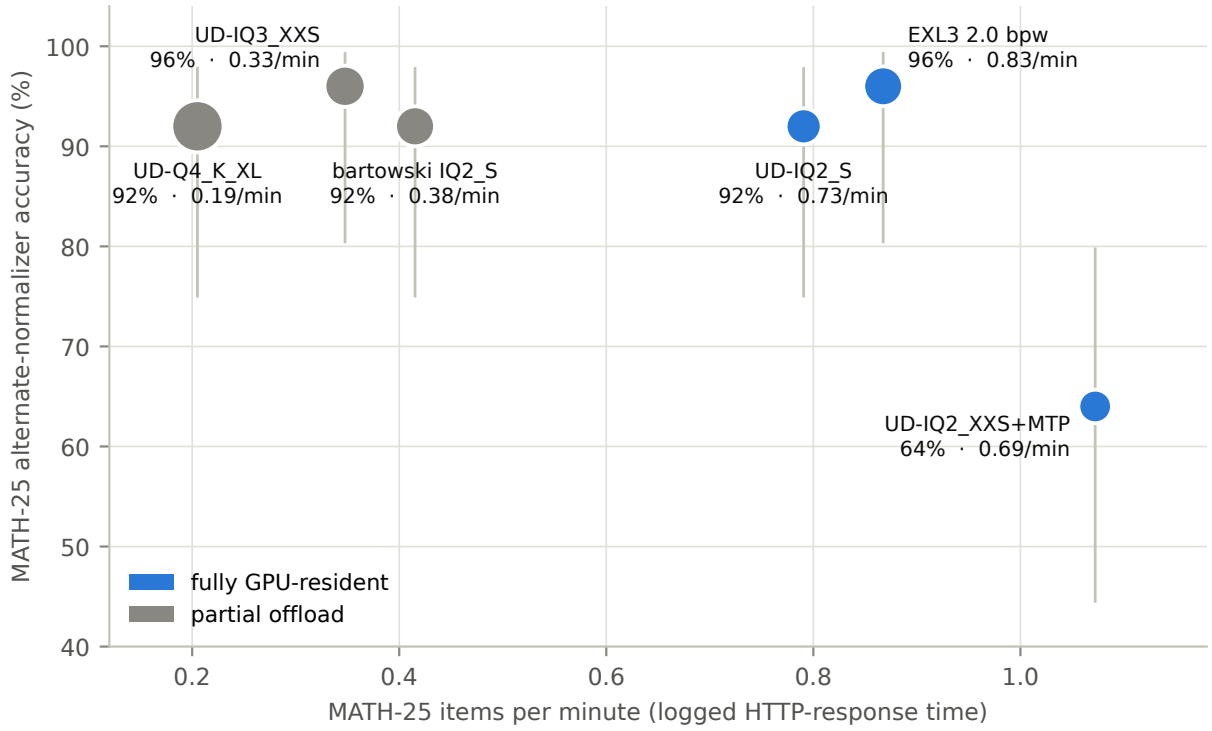

Figure 4: MATH-25 point estimates using logged HTTP-response time. x: benchmark items completed per minute, whether correct or not; y: alternate-normalizer accuracy with Wilson 95% intervals; point area $\propto$ file GiB. Their product is correct responses per minute. Colors encode the measured residency state. The vertical intervals overlap widely; the plot compares observed deployment rates and does not establish quality dominance.

and neither on 14 (exact two-sided McNemar p=0.119). The 3.5-point estimate therefore does not resolve a quality difference, while EXL3 returned more correct items per minute. On the needle probes, `UD-IQ2_S` answered all four 8K/16K contexts but its two 32K items produced no answer after two 600-s HTTP attempts each (one automatic retry after a 2-s delay, or about 1,202 s total per item); EXL3 completed all six. The 4/6 versus 6/6 count measures completion under this client policy and includes a large prompt-processing difference, so it must not be read as two wrong retrieval answers.

3.8 Reasoning-Effort Check

Qwen3.8's chat template exposes a `reasoning_effort` dial. For `UD-IQ2_S`, xhigh produced no observed benefit. On MATH-25 it scored 21/25 (84%) under the alternate normalizer versus 23/25 (92%) at medium, with four rather than one 8K truncations. Mean wall time rose from 75.89 to 134.44 s per item (1.77×). On HumanEval+, an initial xhigh run was truncation-confounded; the 8K-budget re-run removed truncations and scored 16/20 (80%) versus 18/20 (90%) at medium. Neither accuracy difference is resolved at this sample size. The useful conclusion is narrower: xhigh was slower and did not improve either point estimate in these runs, so medium is the better measured setting for this deployment. This is not evidence that additional reasoning effort is generally harmful.

3.9 Correct Responses per Minute at Fixed VRAM

For a fixed suite, correct responses per minute is the number scored correct divided by summed per-item HTTP-response wall time. The timer includes prompt processing, generation, retries, and request overhead, but stops before local post-response parsing or code execution. It answers a deployment question— how many successful model responses this machine returned per minute— without pretending that a correct MATH item and a correct HumanEval+ item have the same value. We therefore compare it only within a suite and always print the underlying accuracy beside it.

The response-time view in Figure 4 avoids the cross-engine token/s problem: it prices prompt processing, generation length, retries, request overhead, MTP, and offload as logged. It does not price local scoring. On MATH-25, EXL3 has the highest observed correct/min (0.83), followed by fully resident `UD-IQ2_S` (0.73) and the faster but less accurate `UD-IQ2_XXS`+MTP (0.69). The partial-offload artifacts range from 0.19 to 0.38. On HumanEval+, EXL3 has the highest observed rate at 2.27, followed by the faster but lower-scoring `UD-IQ2_XXS`+MTP at 1.99 and `UD-IQ2_S` at 1.40. These are efficiency estimates on the observed subsets, not statistically distinct quality rankings or a recommendation that ignores error cost.

4 Discussion

4.1 Deployment Recommendations

For this machine, the measurements support distinct deployment choices. Extrapolation to other 12 GB systems requires repeating the fit search because desktop VRAM, engine buffers, and memory bandwidth change the boundary.

1. **Default when the ExLlamaV3 stack is suitable: EXL3 2.0 bpw (3.03 effective).** It has the highest measured hard-suite correct/min and, among the two configurations selected for validation, the highest MMLU correct/min, with no resolved quality difference from the larger tested anchors. Its speed advantage is workload-dependent, and this run did not configure MTP or vision.
2. **GGUF all-rounder: `UD-IQ2_S`.** It remains fully resident, scored 92%/90% on the hard suites, and uses the llama.cpp/GGUF stack. The separate unsloth MTP draft did not fit in the measured desktop run state.
3. **Maximum throughput when errors are cheaper: `UD-IQ2_XXS`+MTP.** It delivered the fastest measured generation and 6.01 correct GSM8K items/min, but its 64% alternate MATH and 70% HumanEval+ estimates were lower than the resident alternatives. It is not the recommendation for correctness-sensitive work.

4.2 Research Directions

These measurements motivate a broader systems hypothesis: under a fixed VRAM budget, useful single-stream throughput is often maximized by a deployment that sits above the task-accuracy loss boundary but below the engine's offload boundary. The present experiment establishes one instance, not a general law. Testing the hypothesis requires repeating the same fit–speed– accuracy map across models, engines, 8/12/16/24 GB memory budgets, context lengths, and power limits.

The highest-priority quality extension is a bf16, Q8, or hosted reference run on hardware large enough to hold it. That would determine how much accuracy all tested 12 GB deployments lose relative to the original model; the current Q4 anchor only supports comparisons within this experiment. Denser sampling between 2.16 and 2.49 effective bpw, larger hard-task sets, and repeated generations would test whether the observed GGUF drop is stable.

A method-level EXL3 claim would require tighter controls over tensor precision, packaged components, kernels, and memory allocation, ideally with one engine supporting both representations. The MATH audit can be strengthened without rerunning inference: an independent, preregistered human or computer-algebra review can rescore the archived predicted-answer fields. New inference runs would be needed to estimate run-to-run variability or test whether parser compatibility changes systematically with quantization.

5 Limitations and Threats to Validity

- **No unquantized quality reference.** We did not evaluate bf16, Q8, or the hosted model. Comparisons to `UD-Q4_K_XL` rank only the tested artifacts and do not establish parity with the original model.
- **Limited statistical power.** MATH-25 and HumanEval+ contain only 25 and 20 fixed items. Their Wilson intervals are wide; a non-significant comparison is not evidence of equality. MMLU is larger but tests a different capability mix. No correction was applied across the exploratory paired comparisons.
- **One stochastic sample.** Each item was generated once at temperature 1.0. The intervals quantify item sampling only, not variation across repeated generations.
- **Post-hoc alternate scoring.** The MATH normalizer was written after failures were inspected. Publishing its rules, retained per-item scoring records, original scores, and alternate scores makes the judgment auditable but does not make it independent or pre-registered.
- **Deployment, not method isolation.** The EXL3–GGUF comparison changes format, artifact, kernels, memory allocation, and engine together. It supports a choice between the tested deployments, not a causal claim about trellis coding.
- **Speed protocols.** The GGUF ladder uses controlled tg128 runs; no matching EXL3 microbenchmark was collected. Cross-engine ratios in §3.7 instead use engine-reported generation metrics on the same evaluation workloads. Correct/min uses logged HTTP-response wall time and is the more comparable deployment measure; it excludes local scoring time.
- **Scope and provenance.** Results come from one model, laptop, run state, and set of artifacts downloaded on one date. File sizes identify the local artifacts. Upstream revisions were recovered from the local cache after the campaign rather than emitted into the measurement logs at run time; the release records those revisions together with the exact file SHA-256 values. The 80 W power limit is not an energy measurement, and no sustained thermal or per-watt conclusion is reported.
- **Long-context timeout contamination.** The two failed 32K `UD-IQ2_S`+llama.cpp probes exhausted both 600-s HTTP attempts without returning an answer, so the needle comparison combines prompt-processing speed with retrieval behavior.

6 Related Work

Quantization methods. This work evaluates existing post-training quantization rather than introducing a method: GGUF k/i-quants in llama.cpp [4], dynamic mixed-tensor GGUF recipes [13], and EXL3 [7], whose trellis construction builds on QTIP [14]. GPTQ (published at ICLR as OPTQ) [15], AWQ [16], and ParetoQ [17] provide broader calibration and low-bit scaling context. Our effective-bpw accounting is a deployment-footprint measure, not a new quantization metric.

Post-freeze update (checked 2026-08-29). On 2026-08-28, after our 2026-08-25 artifact freeze, ISTA-DASLab released Qwen3.8-27B checkpoints that use GSQ for low-bit scalar quantization and RCO for per-tensor GGUF-type assignment under a file-size budget [18–20]. They are

standard GGUF files, not a GSQ/RCO-specific runtime modification; we did not evaluate their quality, residency, or correct/min on this system.

Evaluation and answer parsing. Prior work shows that aggregate task accuracy can conceal important changes under quantization [21], and low-bit mathematical reasoning has been studied through process and error taxonomies [22, 23]. Separately, structured output research distinguishes semantic correctness from machine usability [24], while evaluation scaffolding can materially alter measured outcomes [25]. Task difficulty and instruction following also interact with quantization [26]. Our MATH audit is a small, concrete instance of these broader warnings: a post-hoc alternate normalizer changed the apparent low-bit gap, so the original-parser and post-hoc-normalized scores must remain visible. Community measurements that report strict or machine-usable output rates provide a related operational view [27].

Extreme-low-bit and cross-format comparisons. Bielik-Q2-Sharp compares six academic extreme-2-bit methods on an 11B Polish model and finds that rankings depend on method and summary metric [28]. Broader unified GGUF evaluation reports FP16, size, perplexity, throughput, and standard downstream tasks on Llama-3.1-8B [29]; a contemporaneous Qwen3.8-27B study compares bf16 through 1-bit GGUFs on larger datacenter GPUs [30]. Those studies provide wider precision or task coverage; the present question is the realized 12 GB deployment boundary. EXL3's documentation reports its own quality curves [7]; an earlier matched-bpw community comparison reported lower MMLU for lower-bpw EXL2 variants than for GGUF at matched quantized-layer bpw on Llama 3 [31]. Our data should not be read as a method-level reversal of either result. At roughly 3 effective bpw we resolve no hard-suite quality difference; the contribution is a matched-file- footprint deployment comparison in which the EXL3 artifact remains resident and the GGUF control does not.

Local inference under resource constraints. Bench360 evaluates quality, latency, energy, and Pareto tradeoffs under a fixed 24 GB budget [32]; other work studies accuracy–performance tradeoffs on server GPUs [33], joint performance, energy, and quality under online GPU serving [34], and various combinations of latency, quality, and energy on mobile, edge, or private-server systems [35–37]. Silicon Showdown describes the consumer "VRAM wall" and the throughput cost of CPU offload [38], while pipelined sub-layer sharding targets that cost with a new client inference system [39].

Most directly, Moreira reports Qwen3.8-27B on a 12 GB RTX 4070 Ti: a fully resident IQ2 configuration generated at 43.643 tokens/s versus 5.977 for a partially offloaded IQ4_XS, while a 24-item custom suite favored IQ4_XS 13–9 without resolving the paired difference [40]. Its desktop hardware, artifact, runtime, context, and speed protocol differ from ours, so those absolute rates are prior evidence about the fit–offload trade rather than a throughput baseline for this study. Thus our study is not the first task-level fit-versus-offload measurement for this model and memory class. It contributes an independent 80 W laptop characterization with standard task families, effective-footprint and engine-residency accounting, a matched-footprint EXL3/GGUF deployment pair, and an explicit parser audit. Correct/min is a descriptive within-suite task-goodput rate, not a proposed universal or novel benchmark.

Our MTP measurements are an instance of speculative decoding [5, 6]; they do not evaluate speculative-decoding algorithms themselves.

7 Artifact Availability and Reproducibility

Every reported result traces to archived JSONL data alongside the harness:

- fit searches, `llama-bench` results, timings, and VRAM peaks in `testsuite/results/phase1.jsonl`;

- per-suite summaries in `testsuite/evals/results/summary.jsonl`; and
- per-item JSONL scoring records in each retained configuration's results directory, including predicted and expected answer fields and the final 200 characters of each saved response where applicable.

The alternate MATH-25 numbers come from the deterministic ∼50-line `testsuite/evals/rescore_math25.py`; every rule is listed in §3.5. Figures are generated directly from these logs by `paper/scripts/make_figures.py`. Engine server logs preserve the per-request metrics used for the matched-workload comparison.

Software: llama.cpp master `b1-1729ed5`, CUDA 12.6.3 (user-local toolchain), SM 8.9, flash-attn on; ExLlamaV3 1.4.3 under TabbyAPI (`max_seq_len` 16384 for the main hard suites and 36864 for validation). Artifacts: unsloth `Qwen3.8-27B-GGUF` UD tier, bartowski `Qwen3.8-27B-GGUF IQ2_S`, turboderp EXL3 `SC_2.00bpw_H3` all downloaded 2026-08-25. File sizes in Table 1 identify the local artifacts; the release manifest records every tested upstream revision, filename, byte count, and SHA-256. Evaluation ran through each engine's OpenAI-compatible endpoint; `reasoning_effort` was passed per request via `chat_template_kwargs`. Harness safety rules (VRAM fit guard including the 1.2 GiB `-ngld 0` draft-buffer correction, mmap-backed CPU weights, one model process at a time) are documented in the harness and were calibrated against the measured peaks in Table 2.

The public artifact should be used in two modes: an analysis-only path that rebuilds tables and figures from archived logs without launching inference, and a prospective replication path that downloads the pinned model artifacts and reruns the GPU harness. The accompanying artifact includes a paper-specific README, environment and hardware requirements, model revisions and checksums, concrete setup and launch commands, expected outputs, and an immutable tag or archive identifier. The version-of-record compendium is archived at doi:10.5281/zenodo.22166977; the matching tagged source and results tree is `https://github.com/matthematics1137/research-artifacts/tree/qwen38-27b-12gb-v1.0.0/papers/2026-qwen38-27b-12gb`. The ML-facing result dataset is `https://huggingface.co/datasets/mv1137/qwen38-27b-12gb-results`. Full response transcripts were not retained: the per-item files contain scoring fields and short answer excerpts. The Q4 easy-suite runs also lack per-item GSM8K and HumanEval files and are supported only by their frozen aggregate summary rows. We preserve these gaps rather than reconstructing records after the fact.

8 Conclusion

On the tested 12 GB laptop, the best measured way to run Qwen3.8-27B was not to load the largest artifact possible and accept CPU offload. Fully resident GGUFs decoded at 24–25 tokens/s, while progressively offloaded GGUFs fell to 16.6–4.9 tokens/s. A 9.50 GiB EXL3 checkpoint remained resident under ExLlamaV3; compared with a 9.59 GiB GGUF under llama.cpp, it generated 1.6× faster on MATH, had no hard-suite score difference resolved at the available sample sizes, and produced the highest observed correct responses per minute on MATH and HumanEval+.

The practical result is that this 12 GB system ran a 27.4B model without the repeated RAM weight reads that dominated the larger GGUF deployments. Finding that operating point required the actual checkpoint footprint, engine memory behavior, GPU residency, response time, and answer parser; the advertised bit label alone was insufficient. This experiment does not establish parity with bf16 or Q8, a universal optimum, or a trellis-specific advantage. Broader claims require larger repeated evaluations, an unquantized or near-lossless reference, and replication across models, engines, and memory budgets.

AI assistance. Claude (Anthropic) and Codex (OpenAI) assisted with evaluation-harness development, experiment execution, evidence auditing, analysis and figure code, and manuscript drafting

and revision. Matthew Schwartz directed the work, reviewed and verified the retained outputs against the archived evidence and cited sources, edited the manuscript, and takes full responsibility for the methods, results, interpretation, citations, and released artifacts.

References

[1] Qwen Team. Qwen3.8-27B model card. Hugging Face model repository, https://huggingface.co/Qwen/Qwen3.8-27B/tree/1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0, 2026. Released 2026-08-14; immutable revision accessed 2026-08-29.

[2] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=r8H7xhYPwz.

[3] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=tEYskw1VY2.

[4] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. GitHub repository commit, https://github.com/ggml-org/llama.cpp/commit/1729ed5371cd1ac6f6d6f3226f8803b080042839, 2026. Exact tested commit; reported build `b1-1729ed5`; accessed 2026-08-29.

[5] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

[6] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023. URL https://arxiv.org/abs/2302.01318.

[7] turboderp-org. ExLlamaV3 and the EXL3 quantization format. Release v1.4.3, https://github.com/turboderp-org/exllamav3/releases/tag/v1.4.3, 2026. Format specification at commit `2398c05635fbbad01a0a51dce63c85c6c8a8450e`, https://github.com/turboderp-org/exllamav3/blob/2398c05635fbbad01a0a51dce63c85c6c8a8450e/doc/exl3.md; accessed 2026-08-29.

[8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. arXiv:2110.14168, 2021. URL https://arxiv.org/abs/2110.14168.

[9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. arXiv:2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.

[10] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html.

[11] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572. Curran Associates, Inc., 2023. doi: 10.52202/075280-0943. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract.html.

[12] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

[13] Unsloth. Unsloth dynamic 3.0 GGUFs. https://unsloth.ai/docs/basics/dynamic-3.0-ggufs, 2026. Tested repository snapshot https://huggingface.co/unsloth/Qwen3.8-27B-GGUF/tree/4ca720788d1e01f1bff70c033e0d0028fd02e502; accessed 2026-08-29.

[14] Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa. QTIP: Quantization with trellises and incoherence processing. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 59597–59620. Curran Associates, Inc., 2024. doi: 10.52202/079017-1904. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/6de2e84b8da47bb2eb5e2ac96c63d2b0-Abstract-Conference.html.

[15] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. OPTQ: Accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=tcbBPnfwxS. Earlier version circulated as "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," arXiv:2210.17323.

[16] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In P. Gibbons, G. Pekhimenko, and C. De Sa, editors, *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html.

[17] Zechun Liu, Changsheng Zhao, Hanxian Huang, Sijia Chen, Jing Zhang, Jiawei Zhao, Scott Roy, Lisa Jin, Yunyang Xiong, Yangyang Shi, Lin Xiao, Yuandong Tian, Bilge Soran, Raghuraman Krishnamoorthi, Tijmen Blankevoort, and Vikas Chandra. ParetoQ: Improving scaling laws in extremely low-bit LLM quantization. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, pages 91311–91336. Curran Associates, Inc., 2025. doi: 10.52202/085713-3055. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/83b17fb3369b1effa97ca5409526b02e-Abstract-Conference.html.

[18] Alireza Dadgarnia, Soroush Tabesh, Mahdi Nikdan, Michael Helcig, Eldar Kurtic, Maximilian Kleinegger, and Dan Alistarh. GSQ: Highly-accurate low-precision scalar quantization for LLMs via gumbel-softmax sampling. arXiv:2604.18556v2, 2026. URL https://arxiv.org/abs/2604.18556v2. Submitted 2026-04-20; revised 2026-05-15.

[19] Michael Helcig and Dan Alistarh. Model compression with exact budget constraints via riemannian manifolds. arXiv:2605.00649v2, 2026. URL https://arxiv.org/abs/2605.00649v2. Submitted 2026-05-01; revised 2026-05-07.

[20] ISTA-DASLab. Qwen3.8-27B: GSQ-RCO GGUFs. Hugging Face model repository, https://huggingface.co/ISTA-DASLab/Qwen3.8-27B-GSQ-RCO-GGUF/tree/0da22c4fad2d0b4d64acdb216789990b69776bac, 2026. Initial repository commit 2026-08-28; accessed 2026-08-29.

[21] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. Accuracy is not all you need. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-3950. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/e0e956681b04ac126679e8c7dd706b2e-Abstract-Conference.html.

[22] Zhen Li, Yupeng Su, Songmiao Wang, Runming Yang, Congkai Xie, Aofan Liu, Ming Li, Jiannong Cao, Yuan Xie, Ngai Wong, and Hongxia Yang. Quantization meets reasoning: Exploring and mitigating degradation of low-bit LLMs in mathematical reasoning. arXiv:2505.11574, 2025. URL https://arxiv.org/abs/2505.11574.

[23] Ekaterina Alimaskina, Darya Rudas, Denis Shveykin, Gleb Molodtsov, Pavel Vasiliev, and Aleksandr Beznosikov. Extreme low-bit inference in reasoning models: Failure modes and targeted recovery. arXiv:2606.02011, 2026. URL https://arxiv.org/abs/2606.02011.

[24] Cosimo Galeone, Minsu Park, Giuseppe Ettorre, and Daniele Ligorio. When correct isn't usable: Improving structured output reliability in small language models. arXiv:2605.02363, 2026. URL https://arxiv.org/abs/2605.02363.

[25] David Gringras. Safety under scaffolding: How evaluation conditions shape measured safety. arXiv:2603.10044, 2026. URL https://arxiv.org/abs/2603.10044.

[26] Jemin Lee, Sihyeong Park, Jinse Kwon, Jihun Oh, and Yongin Kwon. Exploring the trade-offs: Quantization methods, task difficulty, and model size in large language models from edge to giant. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, pages 8113–8121, 2025. doi: 10.24963/ijcai.2025/902. URL https://www.ijcai.org/proceedings/2025/902.

[27] slb350. Strix benchmarks: Quantization insights (Qwen3.5/3.8, Gemma-4 on Strix Halo). https://slb350.github.io/strix-benchmarks/quantization/, 2026. Source commit d890365e8b07ede97fb78925b4345d7f0c4c56b1, src/pages/quantization.astro; accessed 2026-08-29.

[28] Jakub Prejzner. Bielik-Q2-Sharp: A comparative study of extreme 2-bit quantization methods for a polish 11b language model. arXiv:2603.04162, 2026. URL https://arxiv.org/abs/2603.04162.

[29] Uygar Kurt. Which quantization should i use? a unified evaluation of llama.cpp quantization on Llama-3.1-8B-Instruct. arXiv:2601.14277, 2026. URL https://arxiv.org/abs/2601.14277.

[30] Piotr Migdał. Benchmarking Qwen3.8 27B quantizations: 4-bit holds up, 1-bit collapses. Quesma blog, https://quesma.com/blog/qwen38-27b-quantizations-benchmarked/, 2026. Published 2026-08-26; accessed 2026-08-29. The author notes that most tested v2 quant files were later replaced upstream.

[31] matt-c1. Llama-3 quantization comparison (MMLU at matched bits-per-weight, EXL2 vs GGUF). https://github.com/matt-c1/llama-3-quant-comparison, 2024. Commit f35b7db21acd1f6258239774f3dbc0fd6cfbfbae; accessed 2026-08-29.

[32] Linus Stuhlmann, Mauricio Fadel Argerich, and Jonathan Fürst. Bench360: Benchmarking local LLM inference from 360 degrees. arXiv:2511.16682, 2025. URL https://arxiv.org/abs/2511.16682.

[33] Eldar Kurtic, Alexandre Noll Marques, Shubhra Pandit, Mark Kurtz, and Dan Alistarh. "Give me BF16 or give me death"? accuracy-performance trade-offs in LLM quantization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 26872–26886. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1304. URL https://aclanthology.org/2025.acl-long.1304/.

[34] Tianyao Shi and Yi Ding. Systematic characterization of LLM quantization: A performance, energy, and quality perspective. arXiv:2508.16712, 2025. URL https://arxiv.org/abs/2508.16712.

[35] Erik Johannes Husom, Arda Goknil, Merve Astekin, Lwin Khin Shar, Andre Kåsen, Sagar Sen, Benedikt Andreas Mithassel, and Ahmet Soylu. Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency. arXiv:2504.03360, 2025. URL https://arxiv.org/abs/2504.03360.

[36] Yilong Li, Jingyu Liu, Hao Zhang, M. Badri Narayanan, Utkarsh Sharma, Shuai Zhang, Pan Hu, Yijing Zeng, Jayaram Raghuram, and Suman Banerjee. PalmBench: A comprehensive benchmark of compressed large language models on mobile platforms. arXiv:2410.05315, 2024. URL https://arxiv.org/abs/2410.05315.

[37] Alex Khalil, Guillaume Heilles, Maria Parraga, and Simon Heilles. Viability and performance of a private LLM server for SMBs: A benchmark analysis of Qwen3-30B on consumer-grade hardware. arXiv:2512.23029, 2025. URL https://arxiv.org/abs/2512.23029.

[38] Abdurrahman Javat and Allan Kazakov. Silicon showdown: Performance, efficiency, and ecosystem barriers in consumer-grade LLM inference. arXiv:2605.00519, 2026. URL https://arxiv.org/abs/2605.00519.

[39] Aditya Ukarande, Deep Shekhar, Marc Blackstein, and Ram Rangan. Efficient, VRAM-constrained xLM inference on clients. In *Proceedings of Machine Learning and Systems*, volume 8, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/7cd265ae802235b8d5778a4a96ff22dd-Abstract-Conference.html.

[40] Hugo Moreira. Qwen3.8-27B on an RTX 4070 Ti 12GB: A reproducible windows benchmark study. Public evidence release v0.1.0, https://github.com/hugosmoreira/qwen38-27b-rtx4070ti-windows-benchmark/releases/tag/v0.1.0, 2026. Commit d1a60562238c90202bdf431899725f209a418821; accessed 2026-08-29. Discussion: https://huggingface.co/unsloth/Qwen3.8-27B-GGUF/discussions/65.